Lesson 8 Introduction to **CrewAI**

Multi-Agent Orchestration Framework

Agents · Tasks · Crews · Flows

rnal Confidential

What is CrewAI?

CrewAI is the leading open-source framework for orchestrating autonomous AI agents and building complex workflows.

CrewAI Crews

Units of work within your Flow.

Teams of autonomous agents that collaborate to solve specific tasks delegated by the Flow.

Each crew has:

- Specialized agents with defined roles
- Clear tasks with expected outputs
- A coordinated execution process

CrewAI Flows

The backbone of your AI application.

Structured, event-driven workflows that manage state and control execution — the scaffolding for your agents to work within.

Each flow provides:

- Structured state management
- Event-driven execution control
- Coordination of multiple crews

Internal Confidential

Why Use CrewAI?

- Designed for multi-agent collaboration rather than isolated prompt execution
- Separates responsibilities clearly — agents define capability, tasks define work
- Supports structured workflows for complex, multi-step problem solving
- Encourages modular design, making systems easier to reuse and maintain
- Maps well to real-world teamwork, making system design easier to reason about
- Provides a strong foundation for scalable AI applications
- Useful for developing production-oriented multi agent system.

Key Insight

CrewAI turns AI development into a structured system design activity.

Internal Confidential

From Single-Agent Thinking to Multi-Agent Design

Why Do We Need Multi-Agent Systems?

- Many real-world problems are too complex for a single agent.
- A long prompt alone is often not enough to manage complex workflows.
- Large tasks usually involve multiple stages:
  - Information gathering
  - Analysis & synthesis
  - Quality checking
  - Final reporting

- Multi-agent systems divide one large problem into smaller, specialised tasks.
  - Clarity of responsibilities
  - Coordination between Agents
  - Traceability of outputs
  - Workflow organisation
- CrewAI helps developers build these coordinated agent-based workflows more effectively.

Internal Confidential

Core Concepts in CrewAI

01 Agent

Who performs the work

Identity and capability. Each agent has a role, goal, backstory, LLM, and optional tools.

02 Task

What work must be done

Instruction and deliverable. Each task has a description, expected output, and assigned agent.

03 Crew

The coordinated team

Combines agents and tasks into a collaborative unit that can execute a complete workflow.

04 Process

How the workflow runs

The execution logic — sequential or hierarchical — that controls task order.

Internal Confidential

Understanding Agents

An agent represents a specialised AI worker with a defined identity and purpose.

Agent Attributes

- **role**
  - The agent's function or job title
- **goal**
  - The agent's general mission
- **backstory**
  - Expertise and working style that shape behaviour
- **llm**
  - LLM model used by the agent
- **tools**
  - Optional external capabilities

Example Roles

Researcher

Gathers current and relevant information

Analyst

Interprets and draws insights from data

Writer

Composes clear, structured content

Reviewer

Quality-checks outputs before delivery

Agent Configuration — agents.yaml

Agents are defined in YAML for clean separation of configuration and code.

```yaml
researcher:
  role: >
    {topic} Senior Data Researcher
  goal: >
    Uncover cutting-edge developments in {topic}
  backstory: >
    You're a seasoned researcher with a knack for uncovering the latest
    developments in {topic}. Known for your ability to find the most
    relevant information and present it clearly and concisely.
  llm: openai/gpt-4o-mini
```

{topic} Dynamic placeholder — replaced at runtime via kickoff(inputs=…)

llm field Supports OpenAI, Gemini, Claude, and other providers

Internal Confidential

Understanding Tasks

A task defines the exact work assigned to an agent.

Task Attributes

- **description**
  - Detailed instructions for the agent
- **expected_output**
  - The desired final result of the task
- **agent**
  - Which agent performs this task
- **context**
  - Prior task outputs used as input
- **output_file**
  - Optional: save result to a file

Memory rule:
Agent = identity & capability
Task = instruction & deliverable

Key Distinction

Agent goal

Broad and stable
Who the AI is — identity and expertise

Task description

Specific and actionable
What the AI must do right now

Internal Confidential

Task Configuration — tasks.yaml

Tasks are defined in YAML, keeping instructions separate from your Python code.

```yaml
research_task:
  description: >
    Conduct a thorough research about {topic}
    Make sure you find any interesting and relevant information given
    the current year is {current_year}.
  expected_output: >
    A list with 10 bullet points of the most relevant
    information about {topic}
  agent: researcher
```

description

Specific, actionable instructions for the agent to follow

expected_output

Defines exactly what a successful result looks like

agent: researcher

Links directly to the agent defined in agents.yaml

Internal Confidential

How a CrewAI Project Is Organised

agents.yaml

Defines agent identities

role · goal · backstory · llm · tools

tasks.yaml

Defines task instructions

description · expected_output · agent · context

crew.py

Connects agents, tasks, tools, and process

Assembles the full crew for execution

main.py

Launches the system with inputs

Calls .kickoff(inputs={...}) to start

To run the Crew Go to Terminal with the respective Crew project folder, > crewai run

1. Define agent identities

2. Define task instructions

3. Assemble the crew

4. Run via .kickoff()

Internal Confidential

Crew Execution Flow

How Execution Works

- YAML configuration is loaded into the crew
- Agent methods create runtime agent objects
- Task methods create runtime task objects

Crew assembled using a process:

Sequential

- Executes tasks one after another
- Ensures a clear, orderly progression

Hierarchical

- Organizes tasks in a manager–worker structure
- Tasks are delegated
- Requires a manager LLM (manager_llm) or manager agent (manager_agent)
- Manager handles task creation, assignment, and coordination

- Execution begins through .kickoff(inputs=...)

Typical Pipeline

01 **Researcher**

Gathers information on the topic

↓

02 **Analyst**

Interprets and synthesises results

↓

03 **Writer / Reviewer**

Produces and validates final output

Internal Confidential

YAML Configuration Guidelines

1. File Structure

- agents.yaml → config/agents.yaml
- tasks.yaml → config/tasks.yaml

2. Syntax Rules

- Use **2-space indentation**
- Do **not** use tabs
- Top-level keys are **IDs**
- These IDs must match the **Python method names** in your crew class

3. Multi-line Text

- Use > for multi-line values such as:
  - role
  - goal
  - backstory
  - description
  - expected_output
- For short single-line values, use normal key: value

4. Variables

- Use placeholders like {topic}
- Values are passed from Python during kickoff
  Example: crew.kickoff(inputs={'topic': 'AI Agents'})

Internal Confidential

YAML Configuration Guidelines

5. Agents YAML Keys

Required:

- role
- goal
- backstory

Optional:

- llm
- max_iter
- allow_delegation

6. Tasks YAML Keys

Required:

- description
- expected_output

Also includes:

- agent → references an agent ID

Optional:

- output_file
- markdown

7. Tools

Tools are assigned in **Python code**

Tools are **not defined in YAML**

8. Lists

Use the hyphen - syntax with proper indentation context:

```
- research_task
- analysis_task
```

Refer: 1_Creating First Crew Project Steps.pdf to know the complete step by step implementation of your first crew.

Internal Confidential

Best Practices for Designing CrewAI Projects

- Start with simple, well-defined agent roles
- Keep agent goals high-level; make task descriptions specific and clear
- Design every task around an explicit, measurable output
- Maintain consistent naming across YAML files and Python code
- Choose the right workflow pattern — sequential or hybrid — based on the task
- Add tools only when they meaningfully improve agent capability
- Approach CrewAI as workflow engineering, not just prompt writing

Common Mistake

Avoid repeating the same information in both the agent goal and the task description.

Key Message

Effective CrewAI design comes from clear structure, disciplined planning, and purposeful workflow decisions — not trial-and-error prompting.

Internal Confidential

When to Use CrewAI vs OpenAI Agents SDK

✅ Use CrewAI when…

- Multi-agent collaboration is required
- Multiple LLM providers needed (OpenAI, Gemini, Claude…)
- Clear role separation between agents is important
- Sequential, or hybrid workflows
- Primary focus is workflow orchestration
- Designing or teaching AI teams with specialized roles
- Python-focused multi-agent framework.

✅ Use OpenAI Agents SDK when…

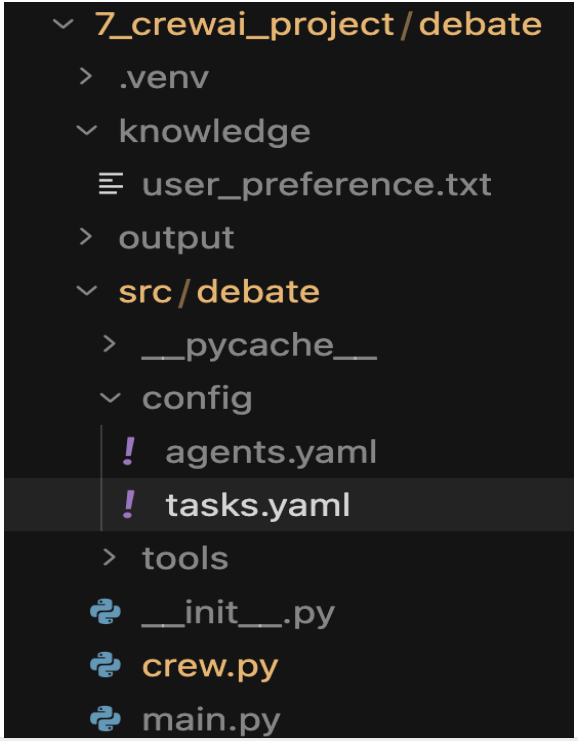
- OpenAI-native agent framework preferred
- Application is centred on agent-tool interaction, handoffs, and guardrails
- Closer alignment with the OpenAI ecosystem
- Design is more focused on agent application development
- Simpler path for OpenAI-specific agent applications
- Python + JavaScript/TypeScript support.

CrewAI → multi-agent workflow orchestration & role-based coordination | OpenAI Agents SDK → OpenAI-native agent application development

Time for the Crewal Installation & First Crew Hands-on

Refer the Demos

✅ Debate Crew



✅ About Debate

Build a simple **CrewAI multi-agent debate system** with two agents: a **Debater** and a **Judge**. The Debater argues **for** and **against** a motion, while the Judge decides the more convincing side. Three tasks—**propose, oppose, and decide**—are defined in tasks.yaml, connected in crew.py, and executed from main.py using kickoff(). The results are saved as **markdown files** for easy review. Save the result as file output_file: output/decide.md

Internal Confidential

What is Task Context?

Automatic Flow

In CrewAI, the output of one task is automatically passed to the next task in sequence.

Explicit Context

Use the `context:` attribute to explicitly define which tasks' outputs are injected as context into the current task.

When you use context=[task], only the task's TaskOutput is passed — not the agent's config, tools, memory, or knowledge.

when you pass context, the Agent Task Output Includes

`description`
Original task description text

`pydantic / json_dict`
Structured output (if configured)

`summary`
Auto-generated from first 10 words

`agent`
Name of the agent that ran the task

`raw`
Actual output produced by the agent

! Role, backstory, goal, knowledge, tools & memory are NOT transferred through context.

YAML Context Syntax & Best Practices

tasks.yaml — Context Syntax

```yaml
your_task_name:
  description: >
    Your task description here.
  context:
    - previous_task_name
    - another_task_name
  expected_output: >
    Your expected result here.
```

← Key attribute

Example Task Execution Flow

`research_task` → `analysis_task` → `report_task`

`analysis_task` uses `context=[research_task]` → waits for and receives its output

Clear Task Names
Use descriptive, readable names so context references are self-documenting.

Order Matters
Always ensure referenced tasks run before the current task.

Circular Dependencies
Avoid circular context references — they will cause a runtime error.

Context Example — research_task → analysis_task

context_example.py PYTHON

```python
From crewai import Task, Agent

research_task = Task(
    description="Research the latest developments in AI",
    expected_output="A list of recent AI developments",
    agent=researcher
)

analysis_task = Task(
    description="Analyze the research findings and
                 identify key trends",
    expected_output="Analysis report of AI trends",
    agent=analyst,
    context=[research_task]  # Waits for output
)
```

1 **Task Definition**

research_task is defined with description, expected_output, and assigned agent.

2 **Context Injection**

analysis_task receives context=[research_task] — its TaskOutput is injected automatically.

3 **Execution Order**

analysis_task waits for research_task to finish before starting.

4 **What Is Passed**

Only the TaskOutput (description, raw, summary...). Agent config is NOT shared.

Output Types — Raw · Pydantic · JSON · File

Raw (default)

str

Plain text string. No special config needed. Default behavior.

result.raw

output_pydantic

BaseModel

Returns a validated Pydantic model. Great for structured, typed data.

output_pydantic=Blog

output_json

dict

Returns a JSON dictionary defined via a Pydantic model.

output_json=Blog

output_file

path

Saves the task output to a file path you specify.

output_file="out.md"

pydantic_output.py

```
from crewai import Agent, Crew, Task
from pydantic import BaseModel
Class Blog (BaseModel):
    title: str
    content: str
task1 = Task(
    description="Create a blog title and content.",
    output_pydantic=Blog,  # ← structured output!
    agent=blog_agent
)
result = crew.kickoff()
print(result["title"]) print(result.pydantic.title)
```

result["title"]
Dictionary-style access

result.pydantic.title
Direct Pydantic attribute

result.to_dict()["title"]
Convert to plain dict

result.raw
Raw string output

verbose=True — Logging & Debugging

verbose=True on Agent

- Logs that individual agent's detailed reasoning
- Shows tool calls and step-by-step actions
- Best for debugging a specific agent

verbose=True on Crew

- Logs end-to-end crew execution trace
- Shows task flow, delegation, overall progress
- Best for a high-level execution overview

verbose_example.py

```
blog_agent = Agent(
    role="Blog Content Generator",
    goal="Generate a blog title and content",
    verbose=True # ← Agent-level verbose
)
crew = Crew(
    agents=[blog_agent],
    tasks=[task1],
    verbose=True # ← Crew-level verbose
)
```

Best Practices

Crew only
Set on Crew for a high-level execution overview.

Agent only
Set on specific Agents to debug individual behavior.

Both = max detail
Use both during development for the most detailed trace.

Both verbose flags default to False. Setting both to True is perfectly fine and recommended during development.

CrewAI Tracing — tracing=True & AMP Platform

tracing=True

A Crew-level parameter that sends execution data to the CrewAI Agent Management Platform(AMP) for monitoring and observability.

What Gets Traced:

- Agent decisions & reasoning
- Task timelines & flow
- Tool usage & calls
- LLM calls & responses

tracing_example.py PYTHON

```python
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    process=Process.sequential,
    tracing=True,  # Covers ALL agents in the crew
    verbose=True
)
```

Setup: Enable AMP Tracing

1. **Sign up at AMP**
   Go to app.crewai.com and create your account.
2. **Authenticate**
   Run crewai login in your terminal.
3. **Enable tracing**
   Set tracing=True on your Crew or Flow object.
4. **Monitor traces**
   Follow the provided link after running your crew.

tracing=True vs verbose=True

Property	Set On	Output Destination	Purpose
tracing=True	Crew	AMP platform (cloud)	Production monitoring & observability
verbose=True	Agent / Crew	Terminal / console	Development debugging & step tracing

Docs: https://docs.crewai.com/en/observability/tracing · Sign up: app.crewai.com

In-Class Activity: Building a CrewAI Workflow

- Design and implement a simple **CrewAI workflow** for an online store.
- Create two collaborating agents:
  - **Product Organizer Agent:** Structures and refines product information.
  - **Marketing Agent:** Converts the structured input into a concise, compelling marketing description.
- Provide inputs:
  - **Product Name**
  - **Target Audience**

Example Input:

- *Product:* Smart Water Bottle
- *Target Audience:* College Students

Objective: Develop a workflow where agents collaborate effectively to transform raw product details into engaging marketing content.

Context: Use context to pass the previous task output to another task.

Internal Confidential

Key Takeaways

01 CrewAI organises AI work around Agents, Tasks, Crews, and Processes — a team-based model that mirrors real-world workflows.

02 Agents define who performs the work; Tasks define what must be done. Keep them clearly separated.

03 YAML-based configuration (agents.yaml, tasks.yaml) encourages clean separation of concerns and reusability.

04 Start simple — 2 agents, 2 tasks, sequential process. Add complexity slowly, then add context, check tracing.

05 Use CrewAI for multi-agent orchestration; use OpenAI Agents SDK for OpenAI-native agent application development.

rnal Confidential

References

- Agents YAML configuration:
  https://docs.crewai.com/concepts/agents#yaml-configuration-recommended

- Tasks YAML configuration:
  https://docs.crewai.com/concepts/tasks#yaml-configuration-recommended

- Add tools to your agents:
  https://docs.crewai.com/concepts/agents#agent-tools

- Add knowledge sources to your crew:
  https://docs.crewai.com/concepts/knowledge#what-is-knowledge

- Crew Process:
  https://docs.crewai.com/en/concepts/processes
  https://docs.crewai.com/en/learn/sequential-process
  https://docs.crewai.com/en/learn/hierarchical-process